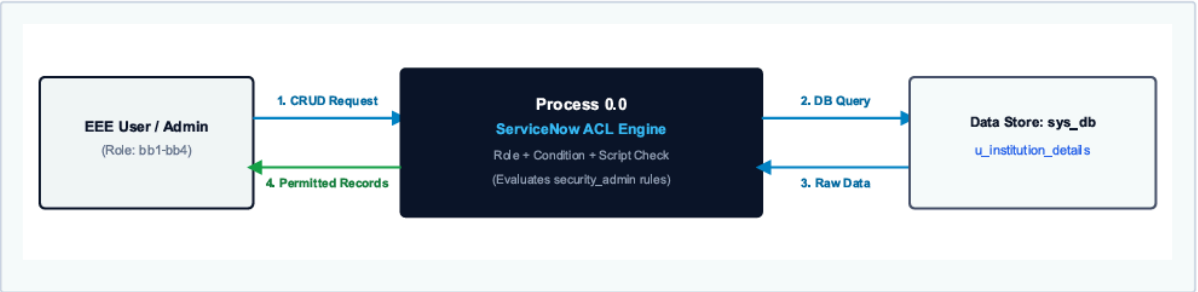
REQUIREMENT ANALYSIS PHASE

DATA FLOW DIAGRAM & USER STORIES

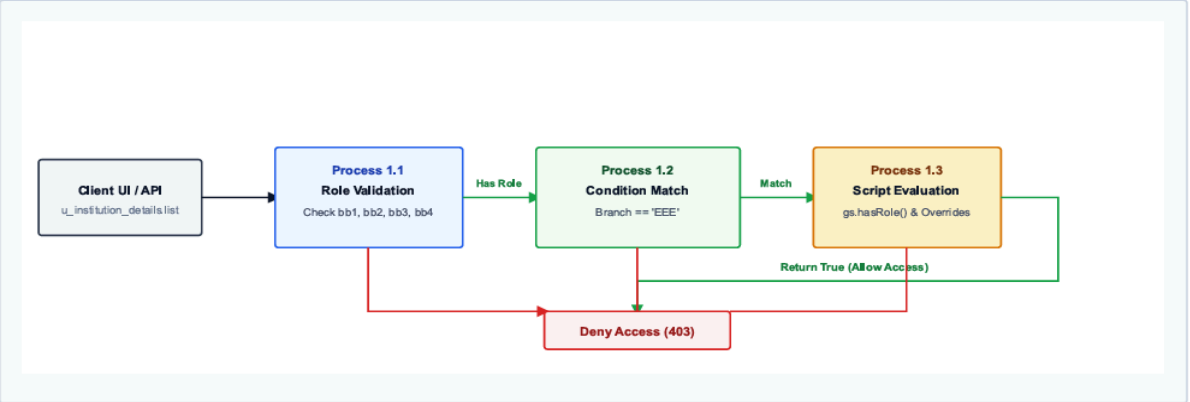
Data Flow Diagram:

The ServiceNow Access Control List (ACL) security engine acts as an enforcement boundary between end users and database tables. For the custom table u_institution_details, security decisions are evaluated at execution time using a three-tier check: Required Roles, Data Conditions, and Scripted Logic.

1.1 Context Level Diagram (Level 0)



1.2 Detailed Data Flow Diagram (Level 1)



1.3 Component Interaction Matrix

COMPONENT	TARGET ASSET	EVALUATED ATTRIBUTE	SYSTEM BEHAVIOR ON FAILURE
Role Matrix	`u_institution_details`	User Roles (bb1–bb4)	Immediate evaluation termination; operational button suppressed in UI.
Data Condition	`u_institution_details.Branch`	Choice value == 'EEE'	Record excluded from GlideRecord result set; row security restriction thrown.
Advanced Script	GlideSystem Context	gs.hasRole('admin')	Returns explicit false; overrides generic table-level access rights.

User Stories:

The following user stories define the operational requirements for data segregation across the institution's branch network (EEE, ECE, CSE). All scenarios follow strict Gherkin syntax (Given-When-Then).

User Story US-01: Branch-Level Data Isolation (Read Operation)
As an EEE Department User, I want to view only institutional records belonging to the EEE branch, so that departmental confidentiality is preserved and unauthorized student data is hidden.
Scenario 1.1:
Authorized Read Access for EEE User Given an active user "EEE User" with assigned role bb1 And the table u_institution_details contains records for branches "EEE", "ECE", and "CSE" When "EEE User" navigates to u_institution_details.list Then the system evaluates the Read ACL rules And displays only records where Branch == "EEE".
Scenario 1.2:
Denied Read Access for Unassigned Role Given a user without the bb1 or admin role When the user attempts to view u_institution_details.list Then the system denies access and displays "No records to display".

User Story US-02: Granular Record Creation Control (Create Operation)
As a Department Administrator, I want to restrict record insertion to users explicitly granted creation rights, so that unverified staff cannot insert invalid student entries.
Scenario 2.1:

Authorized Record Creation Given "EEE User" possesses roles bb1 and bb2 When "EEE User" views the list u_institution_details.list Then the system displays the "New" UI button And allows successful form submission into the database.

Scenario 2.2:

Hidden UI Controls Without Create Role Given "EEE User" possesses role bb1 but lacks bb2 When "EEE User" accesses the table list view Then the system suppresses the "New" UI button.

User Story US-03: Field-Level Update Control (Write Operation)

As an EEE Department User, I want update rights reserved exclusively for authorized write-role holders, so that operational student details cannot be modified by read-only users.

Scenario 3.1:

Authorized Record Editing Given "EEE User" possesses roles bb1, bb2, and bb3 When the user modifies the "Description" field on an EEE record and clicks "Update" Then the system saves the modified values without security exceptions.

User Story US-04: Restricted Record Deletion (Delete Operation)

As a Security Administrator, I want record deletion guarded by an explicit micro-role (bb4), so that accidental or malicious data destruction is prevented.

Scenario 4.1:

Controlled Record Purge Given "EEE User" possesses roles bb1, bb2, bb3, and bb4 When the user selects an EEE record and chooses "Delete" from Actions on selected rows.